
Kế Hoạch Hoàn Thiện Quy Trình UVM (Testcase 1.1 đến 2.3)

Dự án: CVA6 + HPDcache + ICache **Môi trường:** Custom UVM 1.1d cho QuestaSim Starter Edition (No rand/randomize, No SVA, run_phase only)

GIAI ĐOẠN 1: Xây dựng các UVM Agents mới (Yêu cầu bắt buộc từ Testplan)

Để verify Core và ICache, bạn cần bổ sung 2 Agents mới vào file `uvm_systemverilog.txt` (hoặc tạo file package mới `cva6_uvm_pkg.sv`).

1. Tạo `isa_agent` (Dành cho TC 1.1, 1.2, 1.3)

Agent này không dùng để đẩy tín hiệu vào core (vì core chạy bằng mã máy), mà dùng để quan sát và đối chiếu kết quả thực thi lệnh.

- `commit_monitor`: Kết nối với interface RVFI (`rvfi_probes_o`). Bắt các sự kiện khi `commit_valid` lên mức 1. Trích xuất `commit_pc_next`, `commit_instr`, `commit_rd_addr`, và `commit_rd_wdata`.
- `csr_monitor`: (Đặc biệt cho TC 1.3). Bắt các thay đổi trên thanh ghi CSR (đặc biệt là `mepc`, `mcause` khi có exception/bẫy trap).
- `isa_driver` & `isa_sequencer`: Chủ yếu dùng để điều khiển luồng test (ví dụ: kích hoạt tín hiệu reset, báo hiệu test kết thúc) thay vì tạo transaction từng chu kỳ.

2. Tạo `icache_agent` (Dành cho TC 2.1, 2.2, 2.3)

Agent này theo dõi luồng nạp lệnh của ICache.

- `fetch_monitor`:
 - Kết nối với các port Pipeline Request (`dreq_i.req`, `dreq_i.vaddr`).
 - Kết nối với port Response (`dreq_o.valid`, `dreq_o.data`).
 - Giám sát tín hiệu hủy nạp lệnh (kill/flush) như `dreq_i.kill_s1`, `dreq_i.kill_s2` hoặc `fetch_kill_i` (cho TC 2.2).

GIAI ĐOẠN 2: Cập nhật UVM Environment & Scoreboard

1. Cập nhật `tb_env.sv`

- Khai báo và create thêm `isa_agent` và `icache_agent` bên cạnh `hpdcache_agent` hiện có.
- Kết nối các Analysis Port từ `commit_monitor` và `fetch_monitor` vào Scoreboard.

2. Nâng cấp `cva6_scoreboard.sv`

Vì không dùng các bộ Assertion (SVA) phức tạp do giới hạn bản quyền, Scoreboard sẽ đóng vai trò quyết định:

- **Golden Reference Model:** Xây dựng một cơ chế đọc file .hex hoặc mảng dữ liệu (hardcoded golden table) chứa kết quả dự kiến (Expected PC, Expected Reg Write-back).
- **Đối chiếu ALU & Branch (TC 1.1, 1.2):** So sánh dữ liệu từ commit_monitor với bảng Golden Reference. Nếu PC sai lệch hoặc sai giá trị ghi vào thanh ghi Báo UVM_ERROR.
- **Đối chiếu Exception (TC 1.3):** Kiểm tra xem mcause có đúng mã lỗi (ví dụ: Illegal Instruction) và mepc có trỏ đúng địa chỉ sinh lỗi không.

GIAI ĐOẠN 3: Chuẩn bị Mã máy (Firmware) và Xây dựng Testcases

Môi trường không dùng randomize(), do đó kích thích (stimulus) cho Core phải là các chương trình C/Assembly được biên dịch ra file hex (Bare-metal programs).

1. Chuẩn bị Stimulus (Hex files)

Bạn cần viết các đoạn mã Assembly nhỏ, biên dịch bằng RISC-V GCC (riscv64-unknown-elf-gcc) và xuất ra file hex:

- prog_alu.hex: Chứa các lệnh ADD, SUB, AND, OR, SLT.
- prog_branch.hex: Chứa các lệnh BEQ, BNE, JAL, JALR.
- prog_exception.hex: Chứa lệnh ECALL hoặc mã lệnh rác (Illegal).
- prog_fencei.hex: Tự sửa mã lệnh trong bộ nhớ rồi gọi FENCE.I.

2. Xây dựng các Test classes (Tất cả logic đặt trong task run_phase())

Mapping trực tiếp theo file testplan_uvm11d.csv:

Nhóm 1: FULL CORE TEST

- core_basic_alu_ops (TC 1.1): Sequence ra lệnh load prog_alu.hex vào Memory Model Kéo Reset Đợi CPU chạy xong Check UVM_ERROR=0.
- core_branch_prediction_flush (TC 1.2): Load prog_branch.hex Chạy Scoreboard kiểm tra PC nhảy đúng địa chỉ, không có lệnh sai luồng nào được commit.
- core_exception_trap (TC 1.3): Load prog_exception.hex Chạy Bắt event CSR cập nhật mcause hợp lệ.

Nhóm 2: INSTRUCTION L1 CACHE

- icache_seq_fetch_hit_miss (TC 2.1): Core chạy mã tuần tự. Monitor kiểm tra: Lần đọc đầu tiên sinh ra AXI ARVALID (Miss), các lệnh tiếp theo trả về valid=1 liên tục (Hit).
- icache_jump_target_miss (TC 2.2): Monitor bắt các sự kiện tín hiệu kill kích hoạt, ICache hủy nạp lệnh cũ và phát AXI ARVALID cho địa chỉ đích của nhánh nhảy.

- **icache_fence_i_flush (TC 2.3):** Sequence dùng backdoor/AXI Store ghi đè một đoạn mã trong RAM Core gọi FENCE.I Kiểm tra ICache bắt buộc phải issue AXI Read mới thay vì dùng mã cũ lưu trong Cache. (Đây là TC khó nhất, kết hợp cả Data Cache và ICache).

GIAI ĐOẠN 4: Tích hợp vào Testbench Top và Script

1. Cập nhật Interface trong tb_top.sv

Khai báo và liên kết các interface mới vào core:

```
// Interface RVFI cho ISA Agent
cva6_rvfi_if rvfi_vif(.clk_i(clk), .rst_ni(rst_n));
assign rvfi_vif.commit_valid = u_hw.cva6.rvfi_probes_o.commit_valid;
assign rvfi_vif.commit_pc_next = u_hw.cva6.rvfi_probes_o.commit_pc_next;
// ... (tương tự cho các tín hiệu khác)

// Interface ICache cho ICache Agent
cva6_icache_if icache_vif(.clk_i(clk), .rst_ni(rst_n));
// Gán tín hiệu dreq_i, dreq_o, areq_i, areq_o từ ICache

// Truyền vào cấu hình UVM
initial begin
    uvm_config_db#(virtual cva6_rvfi_if)::set(null, "*", "rvfi_vif", rvfi_vif);
    uvm_config_db#(virtual cva6_icache_if)::set(null, "*", "icache_vif", icache_vif);
end
```

2. Cập nhật Script biên dịch (create_full.txt hoặc run_sim.do)

1. Bổ sung đường dẫn compile các file C/Assembly (tùy chọn, hoặc cung cấp sẵn file .hex).
2. Add các file SystemVerilog mới tạo vào luồng compile:
3. Đảm bảo cờ -uvmcontrol=all và -classdebug vẫn được duy trì trong lệnh vsim để hỗ trợ hiển thị Wave cho các components mới.